LAB-05 NOTES – UVM SEQUENCES, CONFIGURATION & SEQUENCE LIBRARIES

1. Goal / Objective

The objective of Lab-05 was to learn how stimulus is generated inside a UVM testbench.

Lab-01 -> Packet
Lab-02 -> Architecture
Lab-03 -> Test & Phases
Lab-04 -> Factory Override

Lab-05 introduces:
• Sequences
• Sequence Inheritance
• Default Sequences
• Config DB
• Active/Passive Agents
• Sequence Libraries

This is where the testbench starts behaving like a real verification environment.

2. What We Had Before Starting Lab-05

Hierarchy available:

top
|
test
|
env
|
agent
|
driver & sequencer

Packet available:
yapp_packet

Missing:
• Traffic Generation
• Directed Stimulus
• Configuration
• Reusable Traffic Patterns

3. Coding Flow Followed

1. Create Base Sequence
2. Create Derived Sequences
3. Learn uvm_do()
4. Learn uvm_do_with()
5. Configure Default Sequence

6. Learn Factory Override with Sequences
7. Learn set_config_int()
8. Configure Active / Passive Agent
9. Create Sequence Library
10. Learn RANDC Selection

4. Step-by-Step Development

Step 1 : Create Base Sequence

class yapp_5_packets extends uvm_sequence #(yapp_packet);

Why?
Need a mechanism to generate packets.

Learning:
Sequence = Traffic Pattern

Step 2 : Learn uvm_do()

Purpose:
Create + Randomize + Start + Send

Syntax:
`uvm_do(pkt)

Learning:
uvm_do() creates random traffic.

Step 3 : Create Directed Sequence

yapp_012_seq

Goal:
Generate packets with addr = 0, 1 and 2.

Syntax:
`uvm_do_with(pkt,{addr_i==0;})

Learning:
uvm_do_with() adds constraints.

Step 4 : Sequence Inheritance

class yapp_1_seq extends yapp_5_packets;

Learning:
Sequences can inherit functionality.

Step 5 : Sequence Composition

yapp_111_seq starts yapp_1_seq three times.

Learning:
One sequence can start another sequence.

Step 6 : Repeat Address Sequence

Store previous address and reuse it.

Learning:
Future transactions can depend on previous transactions.

Step 7 : Incrementing Payload Sequence

Payload:
0 1 2 3 4 ...

Learning:
Not all fields need randomization.

Step 8 : Default Sequence

```
uvm_config_wrapper::set(
this,
"env.agent.sequencer.run_phase",
"default_sequence",
yapp_5_packets::type_id::get()
);
```

Learning:
Tests control traffic generation.

Step 9 : Factory Override With Sequences

Replace:
yapp_packet

With:
short_yapp_packet

Learning:
Factory Override works inside sequences.

Step 10 : Config DB

```
set_config_int(
"env.agent",
"is_active",
UVM_PASSIVE
);
```

Learning:
Parent Writes, Child Reads.

Step 11 : Active vs Passive Agent

Active:
Driver + Sequencer + Monitor

Passive:
Monitor only

Learning:
Active = Drive
Passive = Observe

Step 12 : get_config_int()

Used to read configuration inside build_phase().

Step 13 : Important Discovery

```
super.build_phase();
set_config_int(...);
```

still works because:

create() != build_phase execution

This was one of the most valuable learnings in the lab.

5. Sequence Library

Goal:
Store multiple sequences together.

```
class yapp_seq_lib extends
uvm_sequence_library #(yapp_packet);
```

Added Sequences:
• yapp_012_seq

• yapp_111_seq
• yapp_repeat_addr_seq
• yapp_incr_payload_seq

Learning:
Library = Collection of Traffic Patterns

6. UVM_SEQ_LIB_RANDC

Goal:
Execute every sequence exactly once before repetition.

selection_mode =
UVM_SEQ_LIB_RANDC;

Example order:
111
012
repeat_addr
incr_payload

Learning:

RAND = Dice Roll

RANDC = Deck of Cards

7. Testbench Evolution

Before Lab-05:
Packet
Driver
Sequencer
Agent
Environment
Test

After Lab-05:
Packet
Sequences
Sequence Library
Config DB
Default Sequences
Active/Passive Agent
Factory Override

8. Issues Faced and Solutions

Issue 1:
get_next_item type mismatch

Fix:
uvm_driver #(yapp_packet)
uvm_sequencer #(yapp_packet)

Issue 2:
Too many arguments to new()

Cause:
Used `uvm_component_utils for sequence.

Fix:
Use `uvm_object_utils

Issue 3:
calc_parity not found

Cause:
Function name mismatch:
cal_parity vs calc_parity

Issue 4:
Default Sequence Not Changing

Cause:
Base test overwrote configuration.

Fix:
Apply override after super.build_phase()

Issue 5:
Sequence Library Running One Sequence

Cause:
Only one sequence registered.

Fix:
Register all sequences.

Issue 6:
`uvm_info(..., seq.print(), ...)

Cause:
print() returns void

Fix:
Use seq.sprint()

Issue 7:
Constraint Bug

Wrong:
addr_i == addr_i

Fix:
Store previous address.

Issue 8:
RANDC Confusion

Cause:
Only one sequence in library.

Fix:
Add all sequences to the library.

9. Real Project Importance

Used daily in verification:

• Smoke Sequences
• Directed Sequences
• Random Traffic
• Error Sequences
• Stress Tests
• Regression Libraries

Sequence libraries and configuration mechanisms are heavily used in industry.

10. Interview Questions

Q1. Difference between Sequence and Transaction?

Transaction = Data
Sequence = Traffic Pattern

Q2. Difference between uvm_do and uvm_do_with?

uvm_do = Random Transaction
uvm_do_with = Constrained Transaction

Q3. Active vs Passive Agent?

Active = Drive
Passive = Observe

Q4. RAND vs RANDC?

RAND = Random
RANDC = Random Cyclic

Q5. Why use Sequence Library?

To manage multiple traffic patterns.

11. Final Learning Summary

Packet = Data

Sequence = Traffic

Library = Collection Of Traffic

Config DB = Information Passing

Default Seq = Automatic Traffic

RANDC = Random Without Repetition

Agent:
Active = Drive

Agent:
Passive = Observe

12. One-Line Revision

Lab-05 introduced sequence-based stimulus generation, configuration mechanisms, active/passive agents and sequence libraries, transforming the environment into a fully functional verification framework.

Memory Trick

Lab-01 = Build the Packet
Lab-02 = Build the Roads
Lab-03 = Build the Controller
Lab-04 = Change the Vehicle
Lab-05 = Generate Traffic

Packet exists
Roads exist
Controller exists
Vehicle exists

Now traffic starts moving.

That is exactly what Sequences and Sequence Libraries do.